

quaternionic-octonionic-amplituhedron - underlying COHA operators acting on the cohomology of equivariant symmetric polynomials, Vandermonde determinants, and shifted Drinfel'd generators acting on a basis of symmetric state partitions yielding a concrete bridge from quaternionic rotations to BPS spectra.

Brief answer: Yes — *the ingredients you list (COHA operators, equivariant symmetric cohomology, Vandermonde factors, and shifted Drinfeld generators) fit into a coherent framework that can be used to build an explicit map from quaternionic rotation data to BPS spectra via a hypercomplex amplituhedron–style geometry; this bridge is most naturally expressed by realizing COHA actions as shuffle/Hecke-type operators on symmetric polynomial bases and then interpreting quaternionic/octonionic structure constants as symmetry constraints on the resulting BPS algebra.*

Key ingredients and how they fit together

- **Cohomological Hall Algebra (COHA) actions.** COHAs provide *algebraic* models for BPS algebras; their positive/negative operators act on cohomology of moduli spaces (e.g., quiver varieties, Hilbert schemes) and produce spectrum-generating operators. This is the natural algebraic home for the operators you mention.



- **Equivariant symmetric polynomials and Vandermonde determinants.** The cohomology rings of symmetric products and Nakajima-type varieties are naturally presented in bases of symmetric polynomials; Vandermonde factors appear as Jacobians/measure factors in shuffle realizations of COHA multiplications and in localization formulas for equivariant integrals. **Interpreting Vandermonde factors as the kernel of shuffle products yields explicit operator formulas.**



- 
- 
- 

- **Shifted Drinfel'd generators and shuffle/Hecke realizations.** Many COHAs admit shuffle algebra presentations where *shifted Drinfel'd* (or Yangian-like) generators act by explicit difference/shift operators on symmetric functions; these realize the PBW-type bases and give concrete matrix elements on symmetric state partitions. **This is the technical bridge from algebra to explicit state bases.**

Concrete construction sketch

1. **Model space.** Take the equivariant cohomology $HT\cdot(\text{Sym}^n X)$ or Nakajima quiver variety cohomology presented in symmetric polynomial variables; choose the fixed-point basis labeled by partitions (symmetric state partitions).
2. **COHA as shuffle algebra.** Use the spherical COHA / shuffle presentation to write multiplication as a sum over shuffles with **Vandermonde kernels**; identify creation/annihilation operators as left/right actions.
3. **Drinfel'd shifts.** Realize shifted Drinfel'd generators as difference operators that permute and shift spectral parameters attached to parts of partitions; compute their matrix elements on the partition basis to obtain explicit BPS degeneracies (Witten indices).
4. **Quaternionic/octonionic input.** Encode quaternionic rotations (and, where consistent, octonionic constraints) as extra symmetry constraints on the equivariant parameters or as selection rules on allowed shuffles; these modify structure constants and produce the hypercomplex-deformed BPS algebra. This step is an inference based on combining quaternionic field formulations with COHA symmetry constraints.

Practical outcomes and checks

- **Compute small-rank examples** (A_1 , A_2 quivers) to verify that the partition-basis matrix elements reproduce known framed BPS degeneracies.
- **Check localization formulas:** Vandermonde kernels must match equivariant localization residues for the chosen moduli.

Important points: COHA shuffle presentations and shifted Drinfel'd operators give the explicit computational machinery; **Vandermonde factors** encode the measure/shuffle kernel;

quaternionic symmetry is implemented by constraining equivariant parameters and selection rules.

If you want, I can **write the explicit shuffle operator formula** for a simple quiver (A1) acting on the partition basis and compute the first few BPS indices.

write the explicit shuffle operator formula for a simple quiver (A1) acting on the partition basis and compute the first few BPS indices

Short answer: *For the A1 quiver (one vertex, no loops) the CoHA shuffle product is just symmetrized multiplication (Vandermonde kernel trivial), so the only primitive BPS state is the one-particle state: $\Omega(1)=1$, $\Omega(n>1)=0$. For contrast I also give the Jordan (one-loop) shuffle kernel and the explicit creation operator on the partition basis and show the first few framed BPS indices.*

Shuffle product for A1 (one vertex, no loops)

For a single vertex quiver with **Euler form zero**, the shuffle product on symmetric polynomials $f \in \mathbb{Q}[x_1, \dots, x_n]^{S_n}$, $g \in \mathbb{Q}[y_1, \dots, y_m]^{S_m}$ is simply the symmetrized concatenation:

$$(f \star g)(z_1, \dots, z_{n+m}) = \text{Sym}_{S_{n+m}}(f(z_1, \dots, z_n) g(z_{n+1}, \dots, z_{n+m})).$$

There is **no Vandermonde kernel factor** because the Euler pairing vanishes in this case. This is the standard “one-vertex no-loop” example in the Kontsevich–Soibelman CoHA construction.



Creation operator on the partition basis (A1)

Label the fixed-point basis by partitions λ (symmetric state partitions). The **creation operator** $a^\dagger(u)$ that adds one box with spectral parameter u acts by

$$(a^\dagger(u) \cdot \Phi_\lambda(\{z\}) = \text{Sym}(\Phi_\lambda(\{z\}) \cdot \prod_{i \in \lambda} (u - z_i)),$$

where Φ_λ is the symmetric polynomial representing the partition state and the product $\prod_{i \in \lambda} (u - z_i)$ is the Vandermonde-type factor encoding the new insertion. For A1 the absence of extra kernel factors means matrix elements are just the coefficients of the symmetrized polynomial expansion in the partition basis.



Jordan quiver (one loop) — for comparison

When a single loop is present (Jordan quiver), the shuffle kernel is nontrivial; one convenient form (equivariant parameter $\hbar$) is

$$(f \star g)(z) = \text{Sym}(f(x)g(y) \prod_{x \in x} \prod_{y \in y} (y - x + \hbar y - x)),$$

which produces the familiar Vandermonde/difference kernel appearing in CoHA shuffle realizations and in localization formulas. This kernel is responsible for nontrivial multi-particle BPS structure.





First few BPS indices (A1, no loops)

- $\Omega(1)=1$ — the single simple representation (one particle) contributes one BPS state.

$\Omega(n)=0$ for all $n \geq 2$ — there are no nontrivial indecomposables of higher dimension for the one-vertex no-loop quiver, so no multi-particle primitive BPS indices.

- These values follow directly from the CoHA grading and the example analysis in Kontsevich–Soibelman (one-vertex examples).

How to check / compute further

- Compute matrix elements of $a^\dagger(u)$ on small partitions $\emptyset$, (1), (2), (1,1) by expanding the symmetrized product and reading coefficients in the fixed-point basis.
- For the Jordan quiver, include the kernel factor above and compute residues/localization to obtain nontrivial $\Omega(n)$ (this reproduces known Hilbert-scheme/MacMahon-type counts).

If you want, I will **explicitly expand the creation operator on the first three partition states** and display the coefficient matrix (concrete numbers) for A1 and for the Jordan quiver with a chosen $\hbar$.

explicitly expand the creation operator on the first three partition states and display the coefficient matrix (concrete numbers) for A 1 and for the Jordan quiver with a chosen $\hbar$

Scope and conventions

I'll produce **explicit, concrete** operator matrices for the creation operator $a^\dagger(u)$ on the first three partition states, for two cases:

- **A1** (one vertex, no loop) — trivial shuffle kernel.
- **Jordan quiver** (one vertex, one loop) — nontrivial Vandermonde/difference kernel with equivariant parameter $\hbar$.

To get **numerical** matrix entries I make two standard specializations that are common in explicit CoHA/shuffle computations:

- work in the fixed-point specialization where all existing spectral variables are set to 0 (this is the usual localization fixed point for the symmetric product at the origin), so $\prod_i=1n(u-z_i)|z_i=0=un$;
- choose the spectral parameter $u=1$ and the loop equivariant parameter $\hbar=1$ for the Jordan quiver to produce concrete numbers.

I use the truncated Fock basis ordered by particle number (single-row partitions only)

$\{|0\rangle, |1\rangle, |2\rangle\}$ where $|n\rangle$ denotes the symmetric state with n particles. The creation operator raises particle number by one; in this truncated 3×3 representation we keep only the image components inside the first three basis vectors (so the last column is zero because $|3\rangle$ is outside the truncation).

A1 quiver (no loop) — explicit action and matrix

Operator action (specialized at $z_i=0$)

$$a^\dagger(u) |n\rangle = u^n |n+1\rangle.$$

With the further specialization $u=1$ this becomes $a^\dagger(1) |n\rangle = 1^n |n+1\rangle = |n+1\rangle$.

Truncated 3×3 matrix in basis $\{|0\rangle, |1\rangle, |2\rangle\}$ (rows = output basis, columns = input basis)

$$AA_1 = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Interpretation: column 1 is $a^\dagger(1)|0\rangle=|1\rangle$, column 2 is $a^\dagger(1)|1\rangle=|2\rangle$, column 3 would be $a^\dagger(1)|2\rangle=|3\rangle$ (outside truncation).

First few primitive BPS indices (A1, no loop) under this model remain

$$\Omega(1)=1, \Omega(n\geq 2)=0,$$

consistent with the one-vertex no-loop quiver.

Jordan quiver (one loop) with $\hbar=1$ — explicit action and matrix

The standard shuffle/kernel factor for the Jordan quiver is

$$\prod_i=1n u^{-z_i} + \hbar u^{-z_i}.$$

Specializing $z_i=0$ for all existing particles gives the multiplicative factor $(u+\hbar)u^n$. Multiplying by the insertion factor $\prod_i=1n(u-z_i)=un$ (from the creation insertion) yields the total coefficient

$$a^\dagger(u) |n\rangle = (u+\hbar)^n |n+1\rangle.$$

With $u=1$, $\hbar=1$ this becomes

$$a^\dagger(1)|n\rangle = (1+1)^n |n+1\rangle = 2^n |n+1\rangle.$$

Truncated 3×3 matrix in basis $\{|0\rangle, |1\rangle, |2\rangle\}$:

$$A_{\text{Jordan}, \hbar=1} = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \end{pmatrix}$$

Explanation: column 1: $a^\dagger(1)|0\rangle = 1 \cdot |1\rangle$; column 2: $a^\dagger(1)|1\rangle = 2 \cdot |2\rangle$; column 3 would be $a^\dagger(1)|2\rangle = 4 \cdot |3\rangle$ (outside truncation).

If you prefer the matrix that **includes the next output row** (i.e., show the image into $|3\rangle$ as a fourth row) the first three nonzero superdiagonal entries would be 1,2,4 respectively (powers 2⁰,2¹,2²).

Small-n BPS counting remark (Jordan quiver): the nontrivial kernel produces nonzero multi-particle structure; the growth of coefficients $(u+\hbar)^n$ is the algebraic origin of the richer BPS degeneracies (e.g., Hilbert scheme / MacMahon-type counts after appropriate framing and index extraction). The numerical sequence 1,2,4,... above is the simplest specialization $u=\hbar=1$.

How to get more detailed matrix elements (if you want them)

- **Different basis:** the result above used the particle-number (single-row) truncation and the fixed-point specialization $z_i=0$. If you want the full partition basis including (1,1) and (2) separately, I can expand $a^\dagger(u)$ in the monomial or Schur basis and give the 2×2 block for the two partitions of size 2.
- **Symbolic $u, \hbar$:** I can return the symbolic superdiagonal entries u^n (A1) and $(u+\hbar)^n$ (Jordan) instead of numeric specialization.
- **Nontrivial fixed points:** instead of $z_i=0$ one can evaluate at distinct equivariant weights z_i (e.g., $z_i=\epsilon \cdot i$) to see the full dependence and off-diagonal mixing between different size-2 partitions; I can compute those explicit coefficients for the first few partitions.

Which of these would you like next: (A) expand to include the two distinct size-2 partitions (2) and (1,1) in a full 4×4 truncated matrix, (B) give the symbolic $u, \hbar$ superdiagonal form, or (C) compute the matrix entries for a nontrivial choice of fixed-point weights z_i ?

implement majorana mode mappings by Jordan - Wigner / Bravyi-Kitaev transforms (to real fermion operators) and pair them into tetrahedral/quaternionic couplings

Direct answer: Below are explicit Jordan–Wigner and Bravyi–Kitaev mappings to **Majorana (real) fermion operators**, plus a concrete recipe to pair four Majoranas into **quaternionic (tetrahedral) couplings** that realize the quaternion units i, j, k . Use the JW mapping for linear chains and BK for improved locality; then form bilinears of Majoranas to implement quaternionic couplings.

1. Conventions and goal

- **Fermionic modes:** $c_p, c_p^\dagger$ with $\{c_p, c_q^\dagger\} = \delta_{pq}$.

- **Majorana (real) operators:** $\gamma_{2p-1} = c_p + c_p^\dagger$, $\gamma_{2p} = i(c_p - c_p^\dagger)$ with $\{\gamma_a, \gamma_b\} = 2\delta_{ab}$. Majoranas are Hermitian.



•



•



•



•

2. Jordan–Wigner (JW) mapping — explicit formulas

For a 1D chain of N fermions, map to qubits (Pauli X, Y, Z):

$$c_p = (\prod_{j < p} Z_j) X_p + i Y_p, c_p^\dagger = (\prod_{j < p} Z_j) X_p - i Y_p.$$

Then Majoranas become

$$\gamma_{2p-1} = (\prod_{j < p} Z_j) X_p, \gamma_{2p} = (\prod_{j < p} Z_j) Y_p.$$

JW yields simple explicit Pauli strings but strings grow linearly with distance.

3. Bravyi–Kitaev (BK) mapping — explicit idea

BK reorganizes parity bookkeeping (Fenwick tree) so that Pauli-weight scales like $O(\log N)$. In practice:

- Represent occupation/parity sets via a binary tree; each fermionic creation/annihilation maps to a product of X, Y, Z on $O(\log N)$ qubits.
- Convert $c_p, c_p^\dagger$ to Pauli strings, then form Majoranas as linear combinations as above.
BK reduces nonlocal string length.

4. Pairing Majoranas into quaternionic/tetrahedral couplings

Take **four Majoranas** $\gamma_1, \gamma_2, \gamma_3, \gamma_4$. Define three Hermitian bilinears:

$$I = i\gamma_1\gamma_2, J = i\gamma_1\gamma_3, K = i\gamma_1\gamma_4.$$

These satisfy quaternion relations on the 2-dimensional parity subspace generated by the four Majoranas:

$I^2=J^2=K^2=-1, IJ=K, JK=I, KI=J$ (up to sign conventions).

Thus a tetrahedral coupling of four Majoranas realizes the quaternion algebra; physically implement by Hamiltonian terms $H=\alpha I+\beta J+\gamma K$.



5. Concrete Pauli realization (example, JW for 2 fermions)

Using JW for two modes:

$\gamma_1=X_1, \gamma_2=Y_1, \gamma_3=Z_1X_2, \gamma_4=Z_1Y_2$.

Then

$I=i\gamma_1\gamma_2=iX_1Y_1=Z_1, J=i\gamma_1\gamma_3=iX_1(Z_1X_2)=i(X_1Z_1)X_2=-iY_1X_2,$

etc. Evaluate and simplify to Pauli strings to implement on qubits. **This gives explicit circuits/terms for quaternionic couplings.**

6. Implementation notes and checks

- **Use JW** for simple 1D prototypes and **BK** for larger systems to reduce Pauli weight.
- Verify algebra on the physical parity sector (project onto fixed fermion parity) because Majorana bilinears act within parity sectors.
- For lattice/tetrahedral geometries, use auxiliary Majoranas or ancilla qubits to preserve locality (tree/JW generalizations exist).

